



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences

**Design and Development of an Integrated Data Warehouse
and Data Mining System for E-Commerce Data Warehousing
and Customer Churn Prediction**

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course of

Course Code – CSA1613-Data Warehouse and Data Mining

to the award of the degree of

BACHELOR OF ENGINEERING

IN

Branch Name: CSE

Submitted by

G.Mohammad shareef(192311288)

Under the Supervision of

Dr.K.V. KANIMOZHI

Chennai-602105 Date of

Submission – SEP 2026

Table Of Content

S.NO	Content	Page No
1	Problem Understanding and Business Requirements	3
2	Fundamental Concepts of Data Warehousing	4
3	Data Sources, Integration and ETL Pipeline Design	7
4	Data Preprocessing — Concepts and Techniques	10
5	Data Warehouse Design — Star Schema	11
6	OLAP Concepts and Multidimensional Analysis	13
7	Data Mining Fundamentals and Classification Algorithms	15
8	Model Evaluation, Cross-Validation and Related Concepts	20
9	Model Comparison, Selection and Business Interpretation	22
10	KNIME / Weka Implementation, Visualization and Documentation	23

1. Problem Understanding and Business Requirements

Business context. An e-commerce company operates multiple independent operational systems — a customer relationship system, an order-management system, a product catalogue, a payment gateway, a delivery/logistics tracker, and a website clickstream logger. Management cannot currently see a unified view of buying behaviour, cannot easily identify which products or regions drive revenue, and has no early-warning signal for customers who are about to stop using the platform (churn). The goal of this assignment is to design and build an integrated Data Warehousing and Data Mining solution that solves both problems at once.

1.1 Business Requirements

- Provide a single, consistent, historical view of customers, orders, products, payments, delivery and web interactions for reporting and decision support.
- Support fast multidimensional (OLAP) analysis of sales by time, product, customer, region and payment/delivery channel.
- Identify high-selling products, valuable customer segments, seasonal demand patterns, and regional performance differences.
- Predict which customers are likely to churn, before they churn, so retention actions can be targeted and cost-effective.
- Ensure the solution is scalable, reproducible, and respects customer privacy and fairness.

1.2 Identified Data Sources and Entities

Source System	Key Entities / Attributes	Update Frequency	Format
Customer DB (CRM)	Customer_Id, Name, Gender, DOB, City, Signup_Date, Contact	Daily (CDC)	Relational (MySQL/PostgreSQL)
Order Management System	Order_Id, Customer_Id, Product_Id, Qty, Order_Date, Status	Real-time / hourly	Relational + REST API
Product Catalogue	Product_Id, Name, Category, Sub-category, Brand, Price, Cost	Weekly	Relational / CSV export
Payment Gateway	Payment_Id, Order_Id, Mode, Status, Amount, Timestamp	Real-time	JSON logs / API
Delivery / Logistics	Delivery_Id, Order_Id, Courier, Dispatch/Delivery Date, Status	Daily	CSV / API
Website Interaction	Session_Id, Customer_Id, Page_Views, Cart_Adds, Time_on_Site	Streaming	Clickstream logs (JSON)

1.3 Scope and Deliverables

The solution is organised into three tightly-linked stages: (1) integration of the six source systems above into a dimensional data warehouse through a documented ETL pipeline; (2) multidimensional (OLAP) analysis of the warehouse to surface business insight; and (3) a data-mining stage that trains and compares

classification models to predict customer churn and recommends retention strategies. Section 2 below first lays out the theoretical foundation needed before any of these three stages is implemented.

2. Fundamental Concepts of Data Warehousing

Before designing the ETL pipeline and the schema, it is important to understand the theoretical foundations that justify every design decision made later in this report.

2.1 What is a Data Warehouse?

According to W.H. Inmon, who is widely regarded as the “Father of Data Warehousing”, a Data Warehouse (DW) is a "subject-oriented, integrated, time-variant, and non-volatile collection of data in support of management's decision-making process."

Property	Meaning	Contrast with an OLTP database
Subject-oriented	Organized around major business subjects (customer, product, sales) rather than around applications	OLTP is organized around business processes/functions (order entry, billing)
Integrated	Data pulled from many heterogeneous sources is made consistent (naming, units, encoding)	Each OLTP system has its own local, inconsistent conventions
Time-variant	Every record is associated with a point/period in time; history is preserved	OLTP stores only the current state — old values are overwritten
Non-volatile	Once loaded, data is not updated or deleted in normal operation — only appended	OLTP data is constantly inserted, updated and deleted

2.2 OLTP vs OLAP

Operational systems (the six sources in Section 1.2) are OLTP systems. The warehouse and its analytics layer are OLAP systems. Confusing the two — e.g. running heavy analytical queries directly against the OrderDB — is exactly the problem this assignment's architecture is designed to avoid.

Aspect	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Purpose	Run the day-to-day business (insert an order, update stock)	Analyse the business (trends, comparisons, forecasts)
Data model	Highly normalized (3NF) to avoid update anomalies	Denormalized, dimensional (star/snowflake) for read speed
Typical operation	Short, frequent read/write transactions	Complex, long-running read-only aggregate queries
Data currency	Current data only	Historical, time-variant data
Users	Clerks, cashiers, front-line applications	Analysts, managers, data-mining algorithms
Example query	"Insert this new order"	"What was Q4 revenue by region and category, YoY?"

2.3 Data Warehouse Architecture

A DW is typically built as a layered (tiered) architecture. This assignment implements all three tiers:

1. Bottom tier (Data Source / Staging layer): the six OLTP sources plus a staging area that temporarily holds raw extracted data before cleaning — built in Section 3.
2. Middle tier (OLAP server): the engine that presents data multidimensionally to support fast slicing and dicing — discussed in Section 6 (ROLAP/MOLAP/HOLAP).
3. Top tier (Front-end / Presentation layer): reporting tools, dashboards and the data-mining layer that consume the cube — covered in Sections 7–9.

2.4 Dimensional Modelling: Facts, Dimensions and Measures

- Fact table: stores quantitative, numeric measurements of a business process (e.g. Total_Sales_Amount) together with foreign keys to the dimensions that give that number context. Facts are typically additive (can be summed across all dimensions), semi-additive (summable across some dimensions, e.g. account balances are not summable across time) or non-additive (e.g. ratios, percentages).
- Dimension table: stores the descriptive, textual context that lets a business user filter and group facts — who, what, where, when. Dimension tables are typically wide, shallow, and heavily denormalized (a Star Schema) or partially normalized into sub-dimensions (a Snowflake Schema).
- Measure: a specific numeric column inside the fact table on which an aggregate function (SUM, AVG, COUNT, MIN, MAX) is applied — e.g. SUM(Total_Sales_Amount).
- Grain: the level of detail represented by a single row of the fact table (“one row = one product line item per order” in this design). Choosing the grain is the single most important dimensional-modelling decision — too coarse a grain loses analytical flexibility; too fine a grain increases storage and processing cost.

2.4.1 Schema Types

Schema	Structure	Advantage	Disadvantage
Star Schema	One central fact table directly connected to fully denormalized dimension tables	Simple, fast joins, easy for business users and BI tools to query	Some redundancy inside dimension tables (e.g. Category repeated per product)
Snowflake Schema	Dimension tables are further normalized into sub-dimension tables (e.g. Dim_Product → Dim_Category)	Saves storage, enforces referential integrity in dimensions	More joins → slower queries, more complex for end users
Fact Constellation (Galaxy Schema)	Multiple fact tables share one or more conformed dimensions	Models multiple, related business processes together (e.g. Sales + Web Interaction)	More complex to design and maintain; risk of dimension conformance errors

Design choice justification: this assignment adopts a Star Schema (Section 5) because business-analyst query simplicity and OLAP-tool performance outweigh the storage saving a snowflake would provide at

this data volume; the design can evolve into a Fact Constellation later by adding Fact_Web_Interaction against the same conformed Dim_Time/Dim_Customer/Dim_Location.

2.4.2 Types of Fact Tables

- Transaction fact table: one row per discrete business event (used here — one row per order line item).
- Periodic snapshot fact table: one row per fixed time interval (e.g. daily inventory snapshot), useful for balances/levels that don't have a natural transaction.
- Accumulating snapshot fact table: one row per process instance, updated in place as the process moves through stages (e.g. one row per order updated as it moves Ordered → Paid → Shipped → Delivered).

2.4.3 Types of Dimensions

- Conformed dimension: a dimension (e.g. Dim_Time, Dim_Location) shared identically across multiple fact tables so results can be compared/combined — essential for the Fact Constellation extension mentioned above.
- Degenerate dimension: a dimension attribute with no supporting table, stored directly in the fact table — e.g. Order_Id in Fact_Sales.
- Junk dimension: a single dimension combining several small, low-cardinality flags (e.g. Is_Gift_Wrapped, Is_First_Order) that would otherwise clutter the fact table.
- Role-playing dimension: one physical dimension used more than once in the same fact table with different meaning — e.g. Dim_Time playing the role of both Order_Date and Delivery_Date.

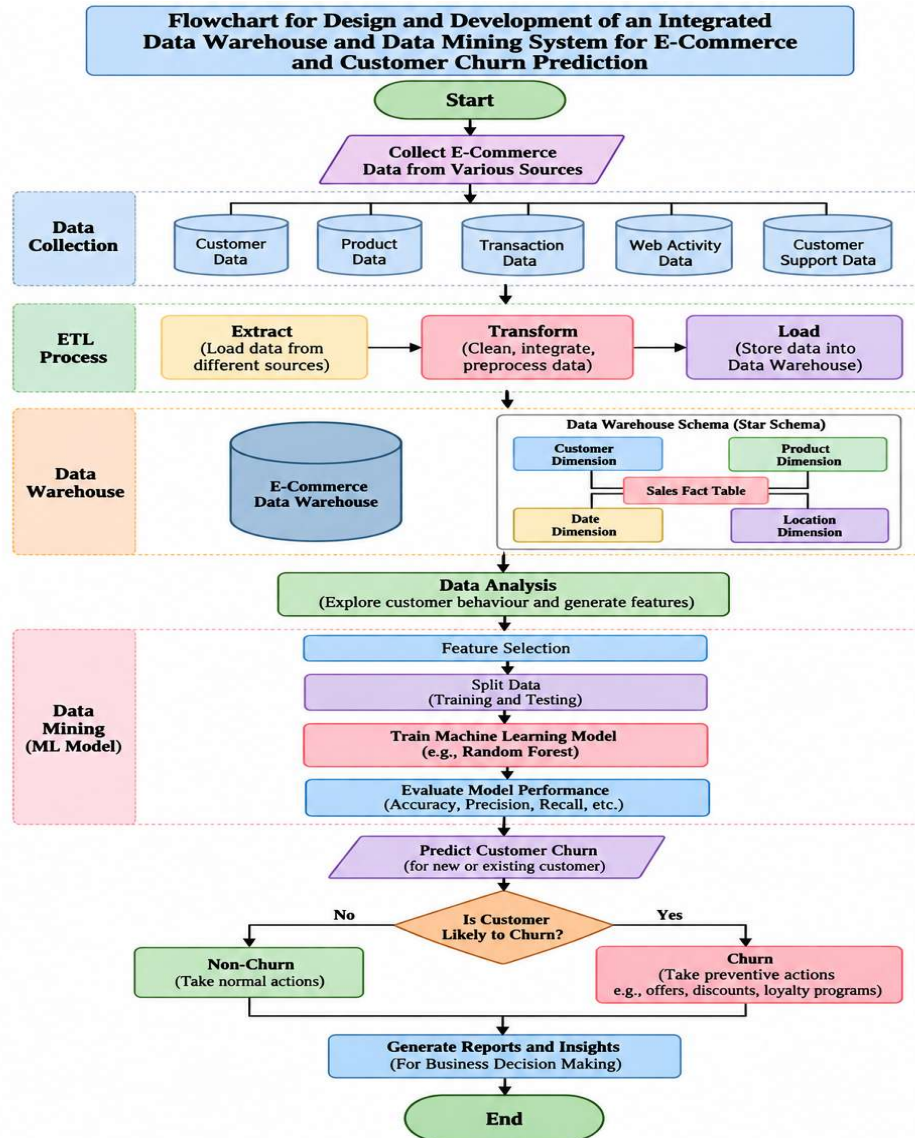
2.4.4 Slowly Changing Dimensions (SCD)

Dimension attributes are not always permanent — a customer's Segment or City can change over time. How the warehouse handles such a change is called a Slowly Changing Dimension strategy:

SCD Type	Behaviour on change	Historical accuracy	Used in this design for
Type 0	Attribute is never updated once loaded	N/A — original value frozen forever	Signup_Date (should never change)
Type 1	Overwrite the old value with the new one	History is lost	Correcting a data-entry error (e.g. misspelled name)
Type 2	Insert a new row with a new surrogate key and validity date range; old row is kept and marked expired	Full history preserved	Dim_Customer.Segment / Churn_Flag (Section 5)
Type 3	Add a new column to store the previous value alongside the current one	Limited history (only one prior value)	Not used here — mentioned for completeness

3. Data Sources, Integration and ETL Pipeline Design

ETL concept. ETL stands for Extract, Transform, Load — the process that moves data from heterogeneous OLTP sources into the dimensional warehouse defined in Section 2. An alternative pattern, ELT (Extract, Load, Transform), loads raw data first and transforms it inside the target platform using its own compute — common with modern cloud warehouses (BigQuery, Snowflake). This assignment uses classic ETL because the transformation logic (SCD Type 2, RFM feature engineering) is easier to version-control and unit-test outside the warehouse.



3.1 Extract

Concept: Extraction reads data out of each source system without altering it. Two extraction strategies exist:

- Full extraction: reads the entire source table every run — simple but expensive; used only for the initial warehouse bootstrap.

- Incremental extraction (Change Data Capture, CDC): reads only rows that changed since the last run, identified via a Last Modified timestamp, a database transaction log, or a trigger-based change table used for all subsequent runs on Orders and Web logs.
- Connect to each of the six sources using the appropriate connector (JDBC for relational DBs, REST client for the payment/API sources, log parser for clickstream JSON).
- Land raw extracts, unmodified, in a staging area (flat files or staging schema) so the source-of-truth is always reproducible and re-runnable.

3.2 Transform

3.2.1 Data Cleaning

Data cleaning fixes noise, missing values, duplicates and inconsistencies the biggest single driver of poor mining-model accuracy if skipped.

- **Missing data handling techniques:**
 - (a) ignore/delete the record (only when the missing attribute is the class label or the loss is negligible)
 - (b) manual fill (impractical at scale)
 - (c) fill with a global constant like “Unknown”
 - (d) fill with the attribute mean/median (numeric) or mode (categorical)
 - (e) fill with the most probable value using regression, decision-tree induction or k-Nearest-Neighbour imputation. This project uses mode/median imputation for Age/City/Delivery_Date and drops only unrecoverable Order_Id nulls.
- **Noisy data / outlier handling techniques:**
 - (a) Binning — sort values into equal-width or equal-frequency bins and smooth by bin mean/median/boundary
 - (b) Regression — fit a function and smooth by predicted values
 - (c) Clustering — group similar values and treat points outside any cluster as outliers
 - (d) statistical thresholding — flag values beyond a Z-score or IQR bound as outliers.
- **Duplicate detection:** exact matching (identical Order_Id) plus fuzzy/approximate matching (edit distance or phonetic matching on Name+Email) to catch near-duplicate customer records created by typos across systems.

3.2.2 Data Integration

Concept: Data Integration combines data from multiple sources into a coherent store. Two classic problems must be solved:

- **Entity Identification Problem:** the same real-world entity (a customer) may have different keys/identifiers in different systems (Customer_Id in CRM vs Email in Payment Gateway). Solved here via a master “crosswalk” key-mapping table and record-linkage rules.

- **Redundancy and Correlation Analysis:** after merging, some attributes may be redundant (derivable from others) or highly correlated, inflating dimensionality without adding information. Detected using χ^2 (chi-square) tests for nominal data and the Pearson correlation coefficient for numeric data.
- **Schema/semantic conflicts must also be reconciled:** naming differences (Cust_ID vs customerId), unit differences (INR vs USD, kg vs g), and structural differences (one system stores Full_Name, another First/Last).

3.2.3 Data Transformation

Transformation converts cleaned, integrated data into the form required by the warehouse and by mining algorithms.

- **Smoothing:** removing noise (see binning/regression above).
- **Aggregation:** summarizing data, e.g. daily clickstream events rolled up to a daily session count — also a data-reduction technique (Section 3.2.4).
- **Generalization:** replacing low-level data with higher-level concepts using a concept hierarchy, e.g. City $\rightarrow$ State $\rightarrow$ Region $\rightarrow$ Country.
- **Normalization:** scaling numeric attributes to a common range so that no single attribute dominates a distance-based algorithm (critical for SVM, k-NN, clustering).
- **Attribute/feature construction:** deriving new, more predictive attributes from existing ones — e.g. Total Sales Amount = Quantity $\times$ Unit Price – Discount;
- Delivery Days = Delivery Date – Order Date; Recency/Frequency/Monetary (RFM) features for churn modelling.
- Encoding categorical attributes: One-Hot Encoding (creates a binary column per category, safe for nominal data with no order, but increases dimensionality) and Label Encoding (maps each category to an integer, compact but implies a false ordinal relationship — acceptable only for genuinely ordinal attributes).

3.2.4 Data Reduction and Discretization

Concept: Data reduction produces a smaller representation of the dataset that still preserves its analytical integrity, reducing mining runtime while maintaining accuracy.

- **Dimensionality reduction:** Principal Component Analysis (PCA) transforms possibly-correlated features into a smaller set of uncorrelated “principal components” ordered by the variance they explain, retaining only the components that capture most of the variance. Attribute subset selection (e.g. stepwise forward selection, correlation filtering) is a simpler alternative that keeps original, interpretable features.
- **Numerosity reduction:** replacing the data with a smaller model or sample — parametric methods (regression models), non-parametric methods (histograms, clustering, sampling). Used here to aggregate raw clickstream events into session-level summaries before they reach the fact table.
- **Data compression:** lossless (e.g. run-length/entropy encoding) or lossy (e.g. wavelet transforms) encoding schemes applied to reduce storage footprint.

- **Discretization:** converting a continuous attribute into a small number of interval labels, which both reduces data and can improve some algorithms' performance (notably Naive Bayes). Methods include equal-width binning, equal-frequency binning, entropy-based (supervised) discretization, and clustering-based discretization. This project discretizes Age into Age Group and Recency into Recency Band using business-defined, equal-frequency bins.
- **Concept hierarchy generation:** organizing discretized values into a hierarchy (Day → Month → Quarter → Year for Dim Time) that directly enables the roll-up/drill-down OLAP operations described in Section 6.

3.3 Load

- Load dimension tables first, applying Slowly-Changing-Dimension Type 2 to Dim Customer (Section 2.4.4) to preserve history of segment/address changes.
- Load the fact table(s) last, resolving foreign keys against the already-loaded dimensions (referential integrity check).
- Run row-count and checksum reconciliation between staging and warehouse after every load; log failures for re-processing.

4. Data Preprocessing — Concepts and Techniques Summary

Why preprocessing matters: real-world data is commonly incomplete (missing attribute values), noisy (containing errors or outliers), and inconsistent (containing discrepancies across sources) — the classic “three data-quality problems” described in data-mining literature. Section 3.2 explained the underlying techniques; the table below consolidates which technique addresses which problem in this specific pipeline, which is what should be justified in the report/reflection.

Technique	Applied To	Method	Justification
Missing value handling	Age, City, Delivery_Date	Mode/median imputation, business-rule defaults	Preserves record volume without introducing bias
Duplicate removal	Customer, Order records	Fuzzy match on name+email; exact match on Order_Id	Prevents double-counting revenue/customers
Outlier treatment	Order_Amount, Delivery_Days	IQR capping ($1.5 \times \text{IQR}$)	Retains extreme-but-valid high-value orders
Normalization	Numeric mining features	Min-Max scaling to [0,1]	Required for distance-based/SVM models (Section 7.4)
Encoding	Categorical attributes	One-hot (nominal), Label (ordinal)	Makes categorical data usable by all three classifiers
Discretization	Age, Recency, Tenure	Equal-frequency / business-defined bins	Improves Naive Bayes performance and interpretability
Dimensionality reduction	Churn feature set	Correlation filter + PCA (optional)	Reduces overfitting risk, speeds up SVM training

4.1 The Data Quality → Model Quality Chain

It is worth stating explicitly why this stage is weighted so heavily in the rubric: every downstream OLAP aggregate and every classification model's accuracy is only as trustworthy as the data feeding it. This principle is summarized by the well-known phrase “Garbage In, Garbage Out” (GIGO) — no algorithm, however sophisticated, can produce reliable churn predictions from uncleaned, inconsistent, or biased input data.

5. Data Warehouse Design — Star Schema

Design choice: Applying the schema theory from Section 2.4.1, a Star Schema (rather than Snowflake) is used as the primary model because query performance and ease of use for business analysts matter more than storage normalisation at this scale; Dim_Product may optionally be snowflaked into Category/Sub-category if the catalogue is very large.

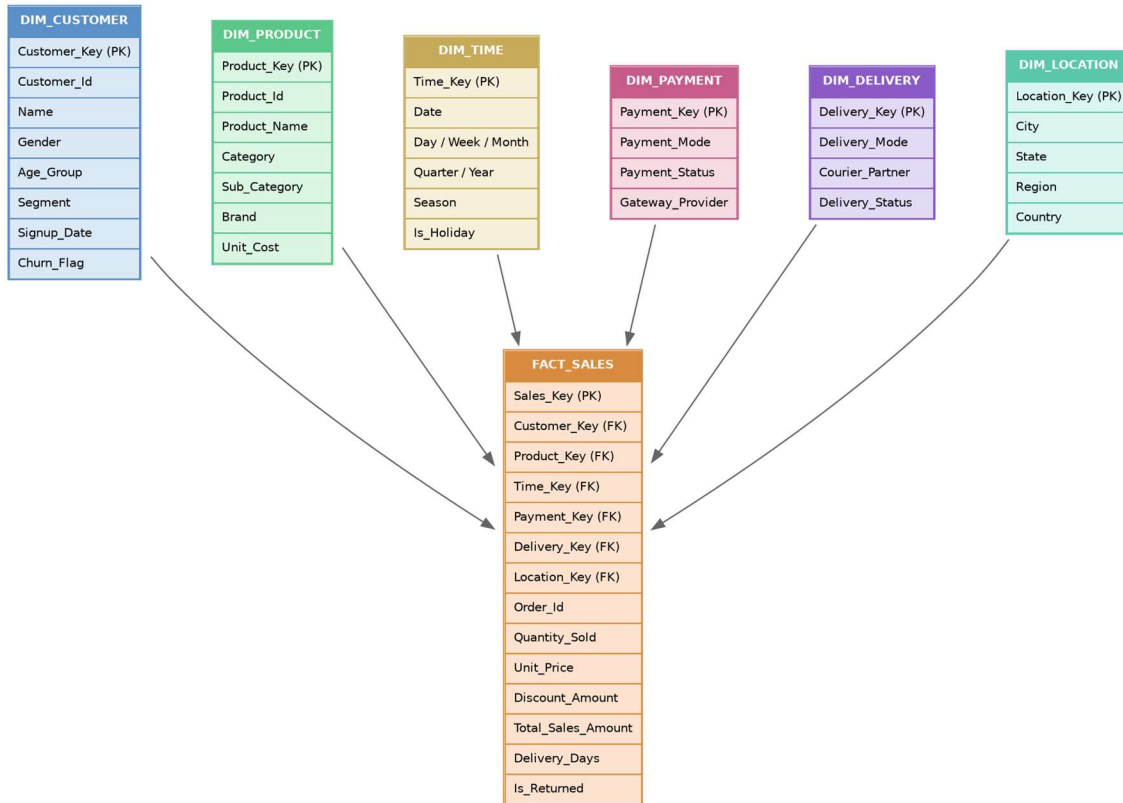


Fig. 2 — Star schema: *FACT_SALES* at the centre with six conformed dimensions

5.1 Fact Table — FACT SALES

Grain: one row per product line-item, per order (Section 2.4, grain concept). Type: a transaction fact table (Section 2.4.2) with additive measures (Quantity Sold, Total Sales Amount, Discount Amount — all summable across every dimension) and one semi-additive measure (Delivery Days, which should be averaged rather than summed across time).

5.2 Dimension Tables

Dimension	Purpose	Notable Attributes	SCD Type
Dim_Customer	Who bought	Segment, Age_Group, Signup_Date, Churn_Flag	Type 2
Dim_Product	What was bought	Category, Sub_Category, Brand, Unit_Cost	Type 1
Dim_Time	When	Day, Week, Month, Quarter, Year, Season, Is_Holiday	Type 0

Dim_Payment	How paid	Payment_Mode, Payment_Status, Gateway_Provider	Type 1
Dim_Delivery	How fulfilled	Delivery_Mode, Courier_Partner, Delivery_Status	Type 1
Dim_Location	Where	City, State, Region, Country	Type 2

5.3 Why this design supports analysis

- Every business question in the assignment (high-selling products, segments, seasonality, regional performance, high-value customers) maps directly to one dimension + the additive measures in Fact Sales, so it can be answered with a single GROUP BY / OLAP cube query — this is the practical payoff of dimensional modelling theory (Section 2.4).
- Conformed dimensions (Dim Time, Dim Location — Section 2.4.3) can be reused by a second fact table (Fact Web Interaction) if clickstream analysis is added later, without redesign, turning the star into a fact constellation.

6. OLAP Concepts and Multidimensional Analysis

6.1 The OLAP Cube

Concept: An OLAP cube (or “data cube”) is a multidimensional array of values, typically constructed from a star schema, where each axis is a dimension (Time, Product, Location, Customer) and each cell holds an aggregated measure (e.g. SUM(Total_Sales_Amount)). Although drawn as a 3-D cube for intuition, the model generalizes to any number of dimensions (a “hypercube”).

6.2 OLAP Server Architectures

Architecture	How data is stored	Query speed	Scalability	Typical use
MOLAP (Multidimensional OLAP)	Pre-aggregated in a proprietary multidimensional array structure	Very fast (pre-computed)	Limited by cube storage — struggles with very high dimensionality (“sparsity”)	Small/medium cubes needing sub-second dashboards
ROLAP (Relational OLAP)	Stored in standard relational tables; aggregates computed on the fly with SQL	Slower (computed at query time)	Scales with the relational engine — handles very large data volumes	Large warehouses (this assignment's Fact Sales)
HOLAP (Hybrid OLAP)	Detail data in relational tables, summary data pre-aggregated in a MOLAP cube	Balanced	Balanced	Warehouses needing both drill-to-detail and fast summaries

6.3 The Five Core OLAP Operations

Build an OLAP cube (or equivalent SQL GROUP BY / CUBE / ROLLUP queries) on Fact Sales with dimensions Time, Product, Customer and Location, and measure Total Sales Amount. Demonstrate and interpret each of the following five operations:

Operation	Definition	Example on this cube	Business Insight Produced
Roll-up	Aggregate from a lower to a higher level of a concept hierarchy (Section 3.2.3), reducing detail	Daily sales → Monthly → Quarterly sales	Reveals seasonal sales patterns and growth trend
Drill-down	The reverse of roll-up: navigate from a higher to a lower level of detail	Region sales → State → City sales	Pinpoints which city drives a region's performance
Slice	Fix one dimension to a single value, view the remaining dimensions as a sub-cube	Sales for Time = “Q4 2025” across all products	Isolates a single period for festive-season analysis

Dice	Select a smaller sub-cube by specifying values/ranges on two or more dimensions	Sales where Region = South AND Category = Electronics	Focused view for a regional-category campaign
Pivot (Rotate)	Rotate the cube's axes to present data from a different cross-tabular viewpoint	Rows = Product Category, Columns = Payment Mode	Shows which categories prefer COD vs online payment

6.4 Analytical Questions the Cube Must Answer

- High-selling products — roll-up Total Sales Amount by Dim Product. Category, rank descending.
- Customer segments — dice by Dim Customer. Segment $\times$ Dim Time. Quarter, compare average order value.
- Seasonal sales patterns — roll-up by Dim Time. Season / Month, plot trend line.
- Regional performance — drill-down Dim Location. Region $\rightarrow$ State $\rightarrow$ City on Total Sales Amount.
- High-value customers — slice on top-decile Total Sales Amount per Customer Key (RFM “Monetary” view, Section 7.1).

Implementation / Source Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

data = {
    "Age": [25, 35, 28, 45, 32, 40, 26, 38, 30, 50,
           29, 42, 33, 48, 27, 36, 31, 44, 24, 39],

    "TotalPurchase": [15000, 5000, 20000, 3000, 18000,
                     4000, 22000, 6000, 16000, 2500,
                     19000, 3500, 17000, 2800, 21000,
                     5500, 17500, 3200, 23000, 4500],

    "NumberOfOrders": [12, 3, 20, 2, 15,
                      3, 25, 4, 14, 1,
                      18, 2, 16, 2, 22,
                      4, 17, 2, 26, 3],
```

```

        "LastPurchaseDays": [10, 90, 5, 120, 15,
                               100, 7, 85, 20, 150,
                               12, 110, 18, 130, 6,
                               95, 14, 115, 4, 80],

        "Churn": [0, 1, 0, 1, 0,
                   1, 0, 1, 0, 1,
                   0, 1, 0, 1, 0,
                   1, 0, 1, 0, 1]
    }

# Convert data into DataFrame
df = pd.DataFrame(data)

print("E-COMMERCE CUSTOMER DATASET")
print(df)

print("\nChecking Missing Values:")
print(df.isnull().sum())

# Remove duplicate records
df = df.drop_duplicates()

print("\nData Preprocessing Completed Successfully")

X = df[[
    "Age",
    "TotalPurchase",
    "NumberOfOrders",
    "LastPurchaseDays"
]]

```

```
y = df["Churn"]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.25,  
    random_state=42  
)
```

```
model = RandomForestClassifier(  
    n_estimators=100,  
    random_state=42  
)
```

```
model.fit(X_train, y_train)
```

```
print("\nRandom Forest Model Training Completed Successfully")
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("\nMODEL ACCURACY:")  
print(accuracy * 100, "%")
```

```
print("\nCLASSIFICATION REPORT:")  
print(classification_report(y_test, y_pred))
```

```
new_customer = pd.DataFrame({
```

```

    "Age": [35],
    "TotalPurchase": [5000],
    "NumberOfOrders": [3],
    "LastPurchaseDays": [90]
})

```

```
prediction = model.predict(new_customer)
```

```
print("\nNEW CUSTOMER PREDICTION:")
```

```
if prediction[0] == 1:
```

```
    print("Customer is likely to CHURN")
```

```
else:
```

```
    print("Customer is NOT likely to churn")
```

Output:

E-COMMERCE CUSTOMER DATASET

	Age	TotalPurchase	NumberOfOrders	LastPurchaseDays	Churn
0	25	15000	12	10	0
1	35	5000	3	90	1
2	28	20000	20	5	0
3	45	3000	2	120	1
4	32	18000	15	15	0
5	40	4000	3	100	1
6	26	22000	25	7	0
7	38	6000	4	85	1
8	30	16000	14	20	0
9	50	2500	1	150	1
10	29	19000	18	12	0
11	42	3500	2	110	1
12	33	17000	16	18	0
13	48	2800	2	130	1
14	27	21000	22	6	0
15	36	5500	4	95	1
16	31	17500	17	14	0
17	44	3200	2	115	1
18	24	23000	26	4	0
19	39	4500	3	80	1

Checking Missing Values:

```

Age          0
TotalPurchase 0
NumberOfOrders 0
LastPurchaseDays 0
Churn        0

```

7. Data Mining Fundamentals and Classification Algorithms

7.1 What is Data Mining? The KDD Process

- Data mining is the process of discovering interesting, non-trivial, previously unknown and potentially useful patterns from large volumes of data. It is one step within the broader Knowledge Discovery in Databases (KDD) process, which consists of the following stages:
- Data Cleaning — remove noise and inconsistent data (Section 3.2.1).
- Data Integration — combine multiple sources (Section 3.2.2).
- Data Selection — retrieve the data relevant to the analysis task from the warehouse.
- Data Transformation — consolidate data into forms appropriate for mining, e.g. aggregation (Section 3.2.3).
- Data Mining — apply intelligent methods (here: classification algorithms) to extract patterns.
- Pattern Evaluation — identify the genuinely interesting patterns representing knowledge, based on interestingness measures (Section 8).
- Knowledge Presentation — visualize and present the mined knowledge to the business user (Section 9–10).
- Mapped to this assignment: Sections 3–4 implement steps 1–4 (the ETL/preprocessing pipeline); this section (7) implements step 5; Section 8 implements step 6; and Sections 9–10 implement step 7.

7.2 Classification as Supervised Learning

Classification is a form of supervised learning: a model (“classifier”) is trained on historical records for which the correct output class is already known (here, Churn Flag = Churned/Active), so that it can predict the class of new, unseen records. This differs from unsupervised learning (e.g. clustering customers into segments with no predefined labels), which is not directly required by this assignment but is a natural extension for the “customer segments” business question in Section 1.

Target: Churn Flag (binary: Churned / Active), defined as no purchase in the last 90 days.

Candidate features (from the warehouse): Recency (days since last order), Frequency (orders in last 12 months), Monetary (Total Sales Amount sum) — together known as RFM analysis, a classic customer-value feature-engineering technique — plus Average Order Value, Tenure Days, Complaint/Return count, Preferred_Category, Preferred Payment Mode, Delivery satisfaction proxy (avg Delivery Days), and Web engagement (avg session time, cart-abandon rate).

7.3 Decision Tree Classifier

Concept: a Decision Tree recursively partitions the training data into purer and purer subsets by repeatedly picking the attribute-test that best separates the classes, producing a tree of if-then decision rules that is easy for a human (e.g. a retention agent) to read directly.

7.3.1 Splitting Criteria

At each node, the algorithm chooses the attribute/split that maximizes class purity in the resulting child nodes. The three classic measures are:

Where p_i is the proportion of records in D belonging to class i . A split with lower resulting entropy/Gini (equivalently, higher Information Gain) is preferred because it makes the child nodes more class-pure. Gain Ratio was introduced by C4.5 specifically to correct ID3's bias toward attributes that simply have many distinct values (e.g. Order Id), which raw Information Gain over-rewards.

7.3.2 Algorithm Variants

Algorithm	Splitting criterion	Handles	Notes
ID3	Information Gain	Categorical attributes only	Original algorithm; biased toward high-cardinality attributes
C4.5 (→ Weka's J48)	Gain Ratio	Categorical and continuous attributes; missing values	Adds post-pruning to reduce overfitting — recommended for this assignment
CART	Gini Index	Categorical and continuous; produces binary splits only	Can also do regression trees

7.3.3 Overfitting and Pruning

A fully-grown tree can perfectly memorize the training data (including its noise), which hurts accuracy on new data this is called overfitting (see Section 8.5). Two families of pruning counter this:

- Pre-pruning (early stopping): halt tree growth early using a threshold (e.g. minimum instances per leaf, maximum depth, minimum information gain).
- Post-pruning: grow the full tree, then remove branches that do not improve validation-set accuracy (e.g. C4.5's error-based / cost-complexity pruning). Generally preferred because it makes decisions with the benefit of seeing the whole tree first.

7.4 Naive Bayes Classifier

Concept: Naive Bayes is a probabilistic classifier based on Bayes' Theorem, with the “naive” (simplifying) assumption of class-conditional independence it assumes the value of one feature (e.g. Preferred Category) provides no information about the value of another (e.g. Payment_Mode) once the class is known. Although this assumption rarely holds exactly in real data, the classifier is remarkably effective in practice and extremely fast to train.

Where $P(C)$ is the prior probability of class C (e.g. proportion of churned customers), $P(x_k|C)$ is the likelihood of feature value x_k given class C (learned from training data as a frequency count for categorical features, or a Gaussian density for continuous features), and $P(X)$ is a normalizing constant that can be ignored since it is the same for every class.

- Laplace (additive) smoothing: prevents a single zero-frequency feature value from making the whole product $P(X|C)$ collapse to zero, by adding a small constant to every count before computing probabilities.

- Variants: Gaussian Naive Bayes (assumes continuous features are normally distributed within each class — suitable for Average_Order_Value), Multinomial Naive Bayes (for count data, e.g. word/event frequencies), Bernoulli Naive Bayes (for binary/boolean features).

Discretizing continuous attributes (Section 3.2.4) before applying Naive Bayes, rather than assuming a Gaussian, often improves accuracy when the true distribution is skewed — which is why Recency and Age are discretized in this pipeline.

7.5 Support Vector Machine (SVM)

Concept: SVM treats classification as a geometric problem: it searches for the hyperplane that separates the two classes (Churned vs Active) in feature space with the maximum margin — the largest possible distance to the nearest training point of either class. The training points that lie exactly on this margin boundary are called support vectors; they alone determine the position of the hyperplane (all other points could be removed without changing it).

- The C (regularization) parameter: controls the trade-off between a wide margin and tolerating some misclassified/slack training points (ξ_i). A small C allows more margin violations (simpler, less overfit boundary); a large C forces fewer violations (tighter fit, higher risk of overfitting).
- The Kernel Trick: real churn data is rarely linearly separable. Instead of explicitly mapping data into a higher-dimensional space where it might become separable (computationally expensive), SVM uses a kernel function $K(x_i, x_j)$ that computes the equivalent of a dot-product in that higher-dimensional space directly from the original features — making non-linear classification computationally tractable.

Kernel	Formula	When to use
Linear	$K(x_i, x_j) = x_i \cdot x_j$	Data is (roughly) linearly separable; fastest; most interpretable
Polynomial	$K(x_i, x_j) = (x_i \cdot x_j + r)^d$	Moderate non-linearity; d controls curve complexity
RBF (Gaussian)	$K(x_i, x_j) = \exp(-\gamma \ x_i - x_j\ ^2)$	Default choice for unknown, non-linear decision boundaries (recommended starting point for churn)
Sigmoid	$K(x_i, x_j) = \tanh(\alpha x_i \cdot x_j + r)$	Behaves like a neural-network activation; less commonly optimal

Because SVM's margin depends on distances between points, feature scaling (Section 3.2.3 normalization) is mandatory before training — an un-scaled attribute like Total_Sales_Amount (range in thousands) would otherwise dominate a small-range attribute like Recency (range in tens).

7.6 Models to Implement (KNIME / Weka)

Model	Algorithm / Node	Key Hyperparameters	Strength for this Problem
Decision Tree	J48 (C4.5) in Weka / Decision Tree Learner in KNIME	Confidence factor, min instances/leaf, pruning	Interpretable rules retention teams can act on directly

Naive Bayes	NaiveBayes node	Kernel estimator on/off, Laplace correction	Fast, robust baseline, works well with discretized features
SVM	SMO (Weka) / SVM Learner (KNIME)	Kernel (linear/RBF), C (regularization), gamma	Strong on high-dimensional, non-linear churn boundaries

7.7 Recommended Workflow

- Load the preprocessed churn feature table (from the warehouse) into KNIME/Weka.
- Partition data 70:30 (train:test), stratified on Churn_Flag to preserve class balance.
- Handle class imbalance (if churners are a minority class): apply SMOTE only — never on the test set, to avoid leaking synthetic information into evaluation.
- Train Decision Tree, Naive Bayes and SVM using identical train/test partitions for a fair comparison.
- Score the held-out test set and export predictions + probabilities for evaluation (Section 8).

8. Model Evaluation, Cross-Validation and Related Concepts

8.1 The Confusion Matrix

A confusion matrix cross-tabulates predicted class against actual class for the test set, and is the basis for every metric below.

	Predicted: Churn	Predicted: Active
Actual: Churn	True Positive (TP)	False Negative (FN)
Actual: Active	False Positive (FP)	True Negative (TN)

In the churn context, an FN (a real churning customer predicted as “Active”) is usually the costlier error — the company loses the customer with zero retention attempt — while an FP (a loyal customer incorrectly flagged) merely costs an unnecessary discount/outreach. This asymmetry is why Recall is emphasized over raw Accuracy in Section 8.3.

8.2 Core Evaluation Metrics

Metric	Why it matters for churn
Accuracy	Overall correctness — can be misleading if churn is rare (a model that always predicts “Active” can still score high accuracy)
Precision	Of predicted churners, how many really churn — controls wasted retention budget
Recall (Sensitivity)	Of actual churners, how many are caught — usually the metric that matters most for retention, per Section 8.1
F1-score	Single number balancing Precision and Recall when both matter
ROC-AUC	Area under the ROC curve (plot of True Positive Rate vs False Positive Rate at every threshold) — measures ranking quality independent of any one threshold, useful for prioritising outreach lists

8.3 The Accuracy Paradox and Class Imbalance

If, say, only 8% of customers churn in a given period, a trivial model that always predicts “Active” achieves 92% accuracy while catching zero real churners — the so-called “accuracy paradox”. This is precisely why Section 7.7 recommends stratified partitioning, SMOTE/cost-sensitive learning, and why Section 8.2 recommends reporting Precision/Recall/F1 rather than accuracy alone whenever the churn class is a minority.

8.4 Cross-Validation

Concept: A single train/test split gives one, potentially lucky-or-unlucky, estimate of model performance. k-fold Cross-Validation gives a more robust estimate by partitioning the data into k equal folds, training on k-1 folds and testing on the remaining fold, repeating this k times (each fold used exactly once as the test set), and averaging the k resulting scores.

- **Stratified k-fold:** preserves the class proportion (e.g. 8% churn) in every fold — recommended here given the likely class imbalance.

- **Leave-One-Out CV (LOOCV):** the extreme case where $k = n$ (every single record is its own test fold); unbiased but computationally expensive, generally impractical at e-commerce data volumes.

8.5 Overfitting, Underfitting and the Bias–Variance Trade-off

Concept	Symptom	Typical cause	Mitigation used in this design
Underfitting (high bias)	Poor accuracy on both train and test data	Model too simple for the pattern (e.g. a depth-1 decision stump)	Increase tree depth / model complexity within reason
Overfitting (high variance)	Near-perfect training accuracy but poor test accuracy	Model memorizes noise/outliers in the training set	Pruning (Section 7.3.3), cross-validation (8.4), regularization (SVM's C, Section 7.5)

8.6 Reporting Template

Model	Accuracy	Precision	Recall	F1-score	10-fold CV Accuracy	Training Time
Decision Tree (J48)	—	—	—	—	—	—
Naive Bayes	—	—	—	—	—	—
SVM (SMO)	—	—	—	—	—	—

Fill this table with the actual figures produced by KNIME/Weka after running the workflow in Section 7, and attach the confusion matrix screenshot for each model as supporting evidence.

9. Model Comparison, Selection and Business Interpretation

Dimension	Decision Tree	Naive Bayes	SVM
Predictive accuracy (typical)	Good, can overfit if unpruned	Moderate, assumes feature independence (Section 7.4)	Often highest on non-linear boundaries (Section 7.5)
Interpretability	High — explicit if-then rules	Medium — probabilistic reasoning	Low — “black box”, especially with RBF kernel
Training speed	Fast	Very fast	Slower, scales poorly with data size
Handles mixed data types	Yes, natively	Needs discretization for continuous features	Needs feature scaling/normalization
Sensitivity to class imbalance	Moderate — improved with cost-sensitive splitting	High — skewed priors bias predictions	Moderate — improved with class-weighted C
Business usefulness	Best for producing retention rules for agents	Best as a fast baseline / sanity check	Best when maximum recall/AUC is the priority

Selection guidance: choose the model with the best Recall/F1 on the churn class (Section 8.1–8.3 explain why missing a real churner is costlier than a false alarm), but weigh this against interpretability — if the retention team needs to understand *why* a customer is flagged, a pruned Decision Tree may be preferred over a marginally more accurate SVM. State your actual selection and justify it using your own experimental numbers from Section 8.

9.1 Recommended Customer-Retention Strategies

- High-risk, high-value customers (top Monetary decile + high churn probability): proactive personal outreach, loyalty discount, dedicated support.
- High-risk, low-engagement customers: re-engagement email/push campaigns, personalized product recommendations from Dim_Product affinity.
- Customers flagged via delivery-related decision-tree rules (e.g. Delivery_Days > threshold): operational fix — prioritise courier partner improvements in that region.
- Customers churning after a complaint/return: fast-track customer-service follow-up within 48 hours of the triggering event.

10. KNIME / Weka Implementation, Visualization and Documentation

This section is a checklist for the practical build — keep it as a running log while implementing, and capture the listed screenshots for the final report.

10.1 Suggested KNIME Workflow Structure

4. File Reader / DB Connector nodes for each of the six sources → Joiner nodes to integrate.
5. Missing Value, Duplicate Row Filter, Outlier node group for cleaning.
6. Rule Engine / Math Formula nodes for derived attributes (Total_Sales_Amount, Delivery_Days, RFM).
7. Normalizer, One-to-Many (one-hot), Auto-Binner nodes for transformation/discretization.
8. Partitioning node (70:30, stratified) → Decision Tree Learner / Naive Bayes Learner / SVM Learner + matching Predictor nodes.
9. Scorer node (per model) → ROC Curve node → Cross-Validation meta-node (X-Partitioner/X-Aggregator, 10-fold).

10.2 Evidence to Capture

- Screenshot of the complete KNIME workflow / Weka Experimenter setup.
- Screenshots of OLAP roll-up/drill-down/slice/dice/pivot results (pivot tables or cube-browser views).
- Confusion matrix and ROC curve screenshot per model.
- Bar/line charts for high-selling products, seasonal trend, and regional performance.
- A short README describing how to re-run the workflow end-to-end.

11. Scalability, Computational Feasibility, Privacy, Ethics and Sustainability

11.1 Scalability and Computational Feasibility

- Incremental (CDC-based) ETL (Section 3.1) keeps nightly load times proportional to new data volume, not total warehouse size.
- Partitioning Fact_Sales by Dim_Time (monthly partitions) keeps OLAP queries fast as history accumulates — exploiting ROLAP's relational scalability (Section 6.2).
- SVM training cost grows faster than Decision Tree/Naive Bayes with data volume (Section 7.5, 9.1 table); consider sampling or a linear kernel if the customer base scales into the millions.

11.2 Privacy and Responsible Data Handling

- Pseudonymise or hash direct identifiers (name, email, phone) before they reach the analytics/mining layer; keep a separate, access-controlled identity-mapping table.
- Apply the principle of data minimisation — only load attributes into the mining feature set that are genuinely predictive, not every available field.
- Define a retention policy (e.g. purge raw clickstream logs after N months) consistent with applicable data-protection regulation.

11.3 Fairness and Ethical Prediction

- Audit churn-model predictions for disparate impact across protected attributes (e.g. gender, age group, region) before deployment.
- Avoid using retention offers that could be perceived as manipulative (e.g. targeting price-sensitive segments with dark-pattern discounts).
- Keep a human-in-the-loop for any retention action with material cost or that could affect a customer negatively (e.g. account restrictions).

11.4 Sustainability (SDG Alignment)

- SDG 8 — improved customer retention supports stable, decent business growth and jobs in the retention/support function.
- SDG 9 — the project itself is an exercise in building resilient, innovative data infrastructure.
- SDG 12 — better demand forecasting from seasonal/regional analysis (Section 6.4) reduces overstocking and returns, supporting responsible consumption.

12. Reflection: Design Decisions, SDG Relevance and Learning Outcomes

12.1 Key Design Decisions and Rationale

- Star schema over snowflake (Section 2.4.1/5): chosen for query simplicity and performance for business users, at the cost of some redundancy in Dim_Product.
- SCD Type 2 on Dim_Customer (Section 2.4.4): needed because customer segment/address changes over time and churn analysis must be historically accurate.
- Recall-weighted model selection (Section 8.3, 9): reflects that, in a churn-retention context, failing to flag a real churner is more costly than a false alarm.

12.2 SDG Relevance

The project connects directly to SDG 8, 9 and 12 (see Section 11.4) — restate this in your own words with a concrete example from your own results once the models are trained.